Mini GDD del microjuego Asteroids – Sebastián Revilla

Fundamentos de Videojuegos · Unity 6000.6.0f1

Microjuego de acción espacial para un jugador, desarrollado sobre el plano XY con gráficos y física 2D. La actividad consiste en maniobrar una nave, destruir meteoritos para obtener puntos y evitar los contactos que reinician la partida.

1 Escenas

SampleScene, guardada en Assets/Scenes/SampleScene.unity, es la única escena incluida y habilitada en EditorBuildSettings. El perfil Windows utiliza esa lista global. La partida comienza directamente en esta escena, sin menú inicial, y vuelve a ella cuando la nave colisiona con un enemigo.

La jerarquía contiene seis objetos raíz: Main Camera, Global Light 2D, Space_0, Ship_0, Canvas y EventSystem. Ship_0 contiene el punto de disparo BulletSpawner, Canvas contiene el marcador Text (Legacy). Los meteoritos y las balas se crean durante la ejecución como objetos independientes. La cámara es fija, ortográfica, está en (0, 0, −10) y tiene tamaño ortográfico 5.

También existen Assets/Settings/Scenes/URP2DSceneTemplate.unity, con cámara e iluminación de plantilla, y Assets/_Recovery/0.unity, con una versión parcial de la escena. Ninguna está incluida en la lista de escenas de la compilación ni se carga desde los scripts del juego.

2 Assets y GameObjects

Assets/Sprites/ aporta los cuatro recursos gráficos utilizados: Ship.png (nave orientada a la derecha), Meteor.png (meteorito con fuego), Bullet.png (proyectil teñido de rojo) y Space.jpg (fondo estrellado). Sus órdenes de dibujo son, respectivamente, 2, 3, 1 y 0. Los sprites usan el material de paquete Sprite-Lit-Default; los niveles de calidad remiten a UniversalRP, con Renderer2D. No hay clips de audio, componentes AudioSource ni animaciones configuradas para el juego.

GameObject	Componentes principales	Función y relaciones
Ship_0	Transform, SpriteRenderer, Rigidbody2D, CapsuleCollider2D, Player	Nave controlada: Etiqueta Player, posición inicial (0, 0, 0) y escala (0,2; 0,2; 1).
BulletSpawner	Transform	Hijo de Ship_0: Punto de salida en posición local (2, 0, 0), asignado a Player.gun.
Bullet_0 (prefab)	Transform, SpriteRenderer, BoxCollider2D, Bullet	Proyectil con collider trigger y sin Rigidbody2D: Referencia bulletPrefab de Player. Escala (3, 3, 1).
Meteor_0 (prefab)	Transform, SpriteRenderer, Rigidbody2D, CircleCollider2D	Enemigo dinámico con etiqueta Enemy y collider sólido: Referencia asteroidPrefab. Escala (0,4; 0,4; 1).
Space_0	Transform, SpriteRenderer	Fondo fijo sin collider, centrado y con escala (2,5; 2,5; 1).
Main Camera	Transform, Camera, AudioListener, datos URP, EnemySpawner	Muestra el campo de juego y aloja la generación de enemigos.
Canvas / Text (Legacy)	RectTransform, Canvas, CanvasScaler, GraphicRaycaster / CanvasRenderer, Text	Interfaz Screen Space Overlay y marcador hijo con etiqueta UI.
Global Light 2D	Transform, Light2D	Luz global blanca de intensidad 1 para los sprites.
EventSystem	Transform, EventSystem, StandaloneInputModule	Infraestructura de eventos de interfaz. No hay botones configurados.

3 Mecánicas

Movimiento e inercia. Player aplica empuje con Rigidbody2D.AddForce en la dirección transform.right y gira el Transform sobre Z. Los parámetros efectivos son thrustForce = 100 y rotationSpeed = 120. La nave tiene masa 1, gravedad 0 y amortiguación lineal 0: conserva la velocidad al soltar el empuje y no tiene un límite de velocidad programado. El empuje negativo permite acelerar hacia atrás, no actúa como un freno específico. La combinación de orientación y velocidad exige anticipar las maniobras.

Cálculo físico. Las entradas se aplican en FixedUpdate, con paso fijo de 0,02 s. El código multiplica tanto el giro como el valor enviado a AddForce por Time.deltaTime: a entrada máxima y con ese paso, el giro solicitado es de 2,4° y la fuerza enviada es de magnitud 2 por actualización física. Por tanto, el valor 100 del Inspector no se aplica directamente como fuerza constante de magnitud 100. Los ejes suavizados también condicionan la respuesta.

Continuidad en los bordes. Al superar los límites, la nave se recoloca en el lado opuesto sin reiniciar su velocidad. Player.Start calcula el límite vertical como tamaño ortográfico + 1, es decir, 6, y el horizontal como 6 × relación de aspecto de la pantalla. La reentrada deja un margen de una unidad respecto al límite opuesto. Con ello se mantiene la navegación en un espacio recurrente, las balas y los meteoritos no utilizan esta lógica.

Disparo. Cada pulsación de espacio crea una instancia independiente de Bullet_0 en BulletSpawner, con rotación identidad. Player sustituye el targetVector inicialmente nulo del prefab por la dirección de la nave en ese instante. El proyectil avanza en línea recta mediante Transform.Translate y caduca a los 3 s. No hereda la velocidad de la nave ni cambia de dirección con ella. No existe disparo continuo por mantener la tecla ni un tiempo de recarga adicional programado.

Aparición y progresión. EnemySpawner, situado en Main Camera, crea Meteor_0 desde la primera actualización que cumple la condición temporal. Aparecen en Y = 7 y en posiciones X enteras entre −12 y 11, la llamada utiliza la sobrecarga entera de Random.Range(-12, 12). La frecuencia inicial es de 30 enemigos/minuto y aumenta una unidad por aparición. Cada intervalo se calcula con la frecuencia anterior al incremento: primero 2 s, después 60/31 s, etc. Esta aceleración gradual aumenta la presión sobre el jugador.

Meteoritos y colisiones. Los meteoritos caen por la gravedad global, con gravityScale = 1, y caducan a los 2 s si no se destruyen antes. No tienen un script propio de desplazamiento. El trigger de una bala detecta la etiqueta Enemy, añade puntuación y solicita destruir ambos objetos. El contacto sólido entre nave y meteorito provoca el reinicio. Estas interacciones usan exclusivamente componentes y eventos de física 2D.

Puntuación y ciclo de partida. Player.SCORE comienza con valor estático 0. Cada evento de impacto válido suma 1 y actualiza el marcador a «Puntos: n», la desaparición por tiempo no puntúa. El texto guardado inicialmente es «Puntos = 0». Al colisionar la nave con un enemigo, el código pone SCORE a 0 y recarga la escena activa, restaurando sus objetos y la frecuencia inicial del generador. No hay contador de vidas, condición de victoria ni pantalla de derrota: el fallo conduce directamente a otra partida. Tampoco hay pausa, reinicio manual, fragmentación o pooling.

4 Scripts

La lógica del microjuego está formada por tres clases MonoBehaviour en Assets/Scripts/. Los parámetros y referencias serializados de SampleScene y los dos prefabs coinciden con los valores descritos en las secciones anteriores.

Script	Responsabilidad	Funcionamiento relevante
Player.cs / Player	Controlar la nave y el ciclo de reinicio.	Start obtiene el Rigidbody2D y calcula los límites desde Camera.main. FixedUpdate lee Thrust y Rotate, aplica empuje y giro. Update recoloca la nave en los bordes y crea proyectiles. OnCollisionEnter2D reinicia los puntos y carga la escena si el otro objeto tiene etiqueta Enemy.
Bullet.cs / Bullet	Desplazar el proyectil, resolver impactos y actualizar puntos.	Start programa su destrucción diferida. Update lo traslada según speed y targetVector. OnTriggerEnter2D filtra Enemy, llama a IncreaseScore y destruye enemigo y bala. UpdateScoreText busca la etiqueta UI y modifica un componente UnityEngine.UI.Text.
EnemySpawner.cs / EnemySpawner	Crear enemigos y aumentar su frecuencia.	Update compara Time.time con spawnNext, calcula el siguiente intervalo, incrementa spawnRatePerMinute y elige la posición de aparición. Instantiate crea el prefab y Destroy programa su caducidad con maxLifeTime.

Relaciones entre clases y objetos. Player utiliza referencias directas del Inspector a BulletSpawner y al prefab Bullet_0. Tras instanciarlo, obtiene su componente Bullet con GetComponent y establece la dirección. EnemySpawner depende únicamente de la referencia al prefab Meteor_0, el motor físico mueve sus instancias sin una clase adicional.

La puntuación acopla Bullet con el campo estático Player.SCORE y con Text (Legacy), localizado mediante la etiqueta UI. Esa etiqueta está asignada al objeto de texto que contiene el componente esperado, no se utiliza TextMeshPro. El reinicio del marcador numérico se realiza en la colisión del jugador, no en Player.Start. Una recarga ajena a esa ruta no garantiza por sí sola el reinicio del dato estático.

Distribución de las actualizaciones. La nave recibe las fuerzas en el paso fijo de física, el movimiento de las balas y la generación de enemigos se evalúan por fotograma. Los callbacks OnCollisionEnter2D y OnTriggerEnter2D conectan la simulación con el reinicio y la puntuación. Instantiate y Destroy gestionan directamente la vida de los objetos, sin reutilización de instancias.

5 Interacción

Acciones del jugador. El proyecto activa Input Manager (Old) (activeInputHandler = 0). Player consulta Input.GetAxis para los ejes personalizados Thrust y Rotate, y Input.GetKeyDown(KeyCode.Space) para disparar. Ambos ejes tienen sensibilidad 3, retorno a cero 3 y snap activado, de modo que la entrada se suaviza.

Entrada	Acción	Resultado
W o flecha arriba	Empuje positivo en Thrust.	La nave acelera hacia su parte frontal según su orientación.
S o flecha abajo	Empuje negativo en Thrust.	Acelera en sentido contrario al frontal, el efecto sobre la velocidad depende del movimiento previo.
A o flecha izquierda	Valor negativo en Rotate.	Giro antihorario alrededor de Z, visto por la cámara.
D o flecha derecha	Valor positivo en Rotate.	Giro horario alrededor de Z, visto por la cámara.
Barra espaciadora	Pulsación de disparo.	Aparece un proyectil rojo en el punto de salida, sigue la dirección capturada al disparar.
Soltar las teclas de movimiento	Los ejes regresan progresivamente a cero.	Cesa gradualmente la orden de empuje o giro, la velocidad lineal adquirida permanece por inercia.

La respuesta del control es visual y física: la nave cambia de orientación y trayectoria, y los disparos aparecen como sprites independientes. El jugador puede observar el resultado de los impactos en la desaparición de los objetos y en el texto del marcador. Este texto es blanco, de tamaño 24, con anclaje central y desplazamiento vertical de −135 píxeles, dentro de un Canvas.

Interacciones automáticas entre objetos. No necesitan una entrada adicional del usuario y se resuelven mediante tiempo, etiquetas y eventos físicos.

Evento	Procesamiento	Respuesta observable
Se cumple spawnNext	EnemySpawner instancia el prefab y programa su caducidad.	Aparece otro meteorito por encima del área visible y cae por gravedad.
La nave rebasa un límite	Player modifica su posición, conservando el Rigidbody2D.	Reaparece en el lado opuesto con su movimiento previo.
Bala contra meteorito	OnTriggerEnter2D reconoce Enemy y actualiza SCORE y Text.	Desaparecen proyectil y enemigo, y el marcador aumenta.
Nave contra meteorito	OnCollisionEnter2D reconoce Enemy y recarga SampleScene.	La nave vuelve al origen y el marcador muestra su texto inicial de cero puntos.
Caduca una bala o un meteorito	Se ejecuta el Destroy diferido.	El objeto desaparece sin modificar la puntuación.

Nave y meteoritos tienen colliders sólidos y Rigidbody2D dinámicos. La bala utiliza un BoxCollider2D trigger, el cuerpo dinámico del meteorito permite la interacción de trigger. Los objetos de juego están en la capa Default, cuya matriz de colisión 2D permite los contactos. La lógica del proyectil ignora objetos sin etiqueta Enemy.